第 6 章　图

6.1 引言

图（graph）是一种比树更为复杂的非线性的数据结构。树中的元素之间具有明确的层次关系，除了根结点，每个元素都有唯一的双亲结点位于其上一层；除了叶结点，每个元素都有若干分支结点在其下一层。而在图中，数据元素之间的关系可以是任意的，也就是对任意一个元素，它既可以有若干前驱，也可以有若干后继，而且任意两个元素之间都可能相关。因此，图可以用来描述很多关系相对复杂的数据结构，比如电路图、人际关系网、网络结点分布、城市地图、公路网络等，图的应用已经渗透到诸如语言学、逻辑学、通信工程、计算机科学和数学等很多领域中。

本章将讨论图论中的几个常见问题。这些问题对应的算法不仅在实践中经常被用到，而且非常有趣，因为在实际应用中，选择适当数据结构可以极大地减少这些算法的运行时间。

6.2 图的定义、基本术语和抽象数据类型

图$G=(V,E)$是由顶点（vertex）集V和边（edge）的集E组成。每条边对应一个点对(v,w)，其中$v,w\in V$。如果图中的点对是有序的，那么该图就是有向图，其中的边也称为弧（arc）；反之称为无向图。

以下是图的基于术语。

- **完全图**：若图中任意两个顶点之间都存在一条边，则该图为完全图。若图中包含n个顶点，那么完全有向图有$n(n-1)$条边，完全无向图则有$n(n-1)/2$条边。
- **邻接点**：若顶点v与w之间存在一条边，则认为顶点v与w邻接。
- **顶点的度**：在无向图中，顶点所连接的边的数目为该顶点的度；在有向图中，以顶点v为起始点的弧的数目成为顶点v的**出度**，而以顶点v为终点的弧的数目称为v的**入度**。
- **子图**：设有两个图$G=(V,E)$和$G'=(V',E')$，若V'是V的子集，即$V'\subseteq V$，且E'也是E的子集，$E'\subseteq E$，则称G'是G的子图。
- **权和网**：图中的每条边都可以对应一个数值，这种与边相关的数值称为权。边上带有权值的图也可以称为网，即有向网或无向网；边上不带有权值，就称为图，即有向图或无向图。
- **路径**：在图G中，顶点v_1到v_k的路径是一个顶点序列$v_1,v_2,\dots,v_k$，使得对于$1\le i<k$，顶点对$(v_i,v_{i+1})\in E$。
- **路径长度**：如果边上不带权值，则路径长度是指路径中包含的边的数目；如果边上带有权值，则路径长度等于路径中包含的边的权值之和。
- **简单路径**：路径中的顶点都是互异的。
- **简单回路**：路径中除了起始点与终止点相同之外，其余顶点互异。
- **连通图和连通分量**：在无向图G中，若两个顶点之间存在路径，则认为这两个顶点是**连通**的。如果在无向图G中，任意两个顶点都是连通的，则称G是连通图。无向图中的极大连通子图称为它的连通分量。显然，如果一个图是连通图，那么它只有一个连通分量，就是它自身；只有非连通图才会有若干连通分量。

- **强连通图和强连通分量**：在有向图中，若从顶点v到w存在路径，则称从顶点v到w是连通的；若在有向图中，任意两个顶点都连通，则称有向图是强连通图；有向图中的极大强连通子图成为它的强连通分量。
- **稀疏图和稠密图**：当图中边的数量比较少（即$e \ll n(n-1)$，其中e表示边的数目，n表示顶点的数目）时，称该图为稀疏图；而当图接近完全图时，称该图为稠密图。

图的抽象数据类型，定义如下：

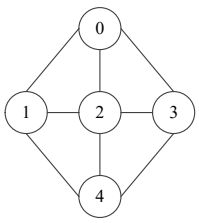
```
ADT Graph
{
    数据对象 V：V 是具有相同特性的数据元素的集合,称为顶点集
    数据关系 R：
        R={E}
        E={<v,w>|v,w∈V 且 P(v,w),<v,w>表示从 v 到 w 的边，谓词 P(v,w)定义了边<v,w>的意义或信息}
    基本操作 P：
        // 1. 初始化和销毁操作
        CreateGraph( &G, V, E )
            初始条件：V 是图的顶点集，E 是图的边的集
            操作结果：按照 V 和 E 的定义构造图 G
        DestroyGraph( &G )
            初始条件：图 G 存在
            操作结果：销毁图 G
        // 2. 访问型操作
        GetVex( G, v )
            初始条件：图 G 存在，v 是 G 中某个顶点（编号/指针）
            操作结果：返回顶点 v 的元素值
        FirstAdjVex( G, v )
            初始条件：图 G 存在，v 是 G 中某个顶点。
            操作结果：返回 v 的第一个邻接点，若 v 没有邻接点，则返回“空”
        NextAdjVex( G, v, w )
            初始条件：图 G 存在，v 是 G 中某个顶点，w 是 v 的邻接点。
            操作结果：返回 v 相对于 w 的下一个邻接点,若 w 已经是 v 的最后一个邻接点,则返回“空”
        DFSTraverse( G )
            初始条件：图 G 存在
            操作结果：对图做深度优先遍历，遍历过程中对每个顶点访问且仅访问一次
        BFSTraverse( G )
            初始条件：图 G 存在
            操作结果：对图做广度优先遍历，遍历过程中对每个顶点访问且仅访问一次
        // 3. 加工型操作
        InsertVex( &G, v )
            初始条件：图 G 存在
            操作结果：在图中添加新的顶点 v
        InsertArc( &G, v, w )
            初始条件：图 G 存在，v 和 w 是图中两个顶点
            操作结果：在图中添加新的边<v, w>，如果是无向图，还应该增加一条对称的边<w,v>
        DeleteVex( &G, v )
            初始条件：图 G 存在，v 是图中顶点
            操作结果：删除图中的顶点 v 及其相关的边
        DeleteArc(&G, v, w )
            初始条件：图 G 存在，v 和 w 是图中两个顶点
            操作结果：删除图中的边<v, w>，如果是无向图，还应该删除一条对称的边<w,v>
} // ADT Graph
```

6.3 图的存储方式

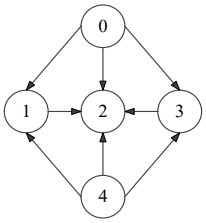
图有多种存储方式，其中最基本的两种存储方式为顺序存储的邻接矩阵和链式存储的邻接表。

6.3.1 邻接矩阵

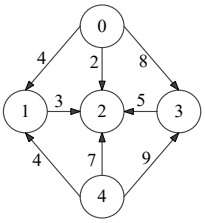
表示图的一种简单方式是使用二维数组，称为邻接矩阵（adjacency matrix）表示法。对于图中的每条边(v,w)，设置$A[v][w]=1$；若不存在边(v,w)，则$A[v][w]=0$。如果边上带有权值，那么可以设置$A[v][w]$等于该权值，同时使用一个很大或者很小的权值来表示不存在的边。如图 6.1 展示了无向图、有向图、有向网和它们对应的邻接矩阵。



a)无向图



b)有向图



c)有向网

$$A=\begin{bmatrix}0&1&1&1&0\\1&0&1&0&1\\1&1&0&1&1\\1&0&1&0&1\\0&1&1&1&0\end{bmatrix}$$

d)无向图的邻接矩阵

$$A=\begin{bmatrix}0&1&1&1&0\\0&0&1&0&0\\0&0&0&0&0\\0&0&1&0&0\\0&1&1&1&0\end{bmatrix}$$

e)有向图的邻接矩阵

$$A=\begin{bmatrix}\infty&4&2&8&\infty\\\infty&\infty&3&\infty&\infty\\\infty&\infty&\infty&\infty&\infty\\\infty&\infty&5&\infty&\infty\\\infty&4&7&9&\infty\end{bmatrix}$$

f） 有向网的邻接矩阵

图 6.1 图的邻接矩阵表示法

邻接矩阵是图的一种顺序存储结构，从邻接矩阵的行数或者列数可知图的顶点数。无向图的邻接矩阵总是对称的，但有向图的邻接矩阵不一定对称。

图的邻接矩阵存储结构的类型声明如下：

```
#define INFINITY INT_MAX        // 定义最大值∞
#define MAX_V  20               // 最大顶点数目
typedef struct
{
    int code;                   // 顶点编号
    ElemType info;              // 顶点其它信息
}VertexType;                    // 顶点类型定义
typedef struct
{
    int arcs[MAX_V][ MAX_V];    // 邻接矩阵
    int vexnum,arcnum;          // 图包含的顶点数与边的个数
    VertexType vexs[MAX_V];     // 存放顶点信息
    GraphKind type;             // 图的种类标志，分为无向图、有向图、无向网和有向网
} MGraph;                       // 声明图的邻接矩阵类型定义
```

以不带权值的图为例，创建以邻接矩阵表示的无向图的算法实现如下。

算法 6.1 基于邻接矩阵表示法的无向图的创建

```
status CreateGraph( MGraph &G )
{
    scanf( &G.vexnum, &G.arcnum );  // 输入图的基本信息，包括顶点与边的数目
    for( i=0; i<G.vexnum; i++ )
        scanf(&G.vexs[i]);                  // 输入顶点信息
    for( i=0; i<G.vexnum; i++ )
        for( j=0; j<G.vexnum; j++ )
            G.arcs[i][j]=0;                 // 邻接矩阵初始化，所有元素初始值为 0(如果是网，则初始值应为极大值)
    for( k=0; k<G.arcnum; k++ )
    {
        scanf(&i,&j);           // 输入一条边，i、j 表示边的两个顶点的编号
        G.arcs[i][j]=1;         // 为邻接矩阵的相应元素赋值
        G.arcs[j][i]=1;         // 无向图中存在一条对称的边也赋值（注意在有向图中不一定存在对称的边）
    }
    return OK;
}//CreateGraph
```

借助图的邻接矩阵很容易判定任意两个顶点之间是否存在边，也容易求得各个顶点的度。

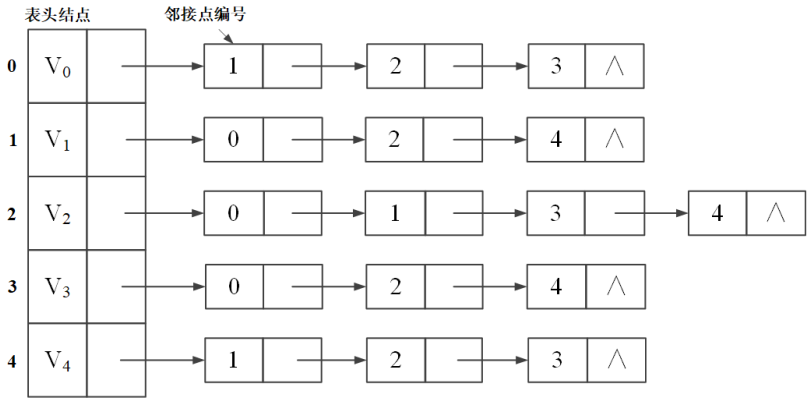
对于无向图，顶点v_i的度是邻接矩阵中第i行的元素之和；对于无向网，顶点v_i的度是邻接矩阵第i行中数值小于极大值的元素个数。

而对于有向图，顶点v_i的出度是矩阵中第i行的元素之和，入度是矩阵中第i列的元素之和；对于有向网，顶点v_i的出度是矩阵第i行中小于极大值的元素个数，入度是矩阵第i列中小于极大值的元素个数。

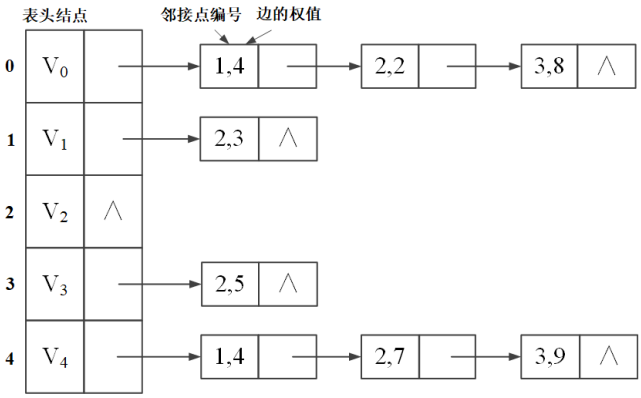
6.3.2 邻接表

采用邻接矩阵表示图的优点是非常简单，但是它的空间复杂度为$O(n^2)$，n表示图中顶点的数目。若图是稠密图，邻接矩阵是合适的表示方法；但如果图是稀疏图，那么这种表示法的代价就太大了。此时，一种更好的解决方法是使用邻接表（adjacency list）表示图。

邻接表是图的一种链式存储结构。它用n个带头结点的单链表代替邻接矩阵的n行，并对图中的每个顶点v建立一个带头结点的单链表，将顶点v的相关信息存放在表头，表中其余的结点用来存放与顶点v相关的边的信息，例如其邻接点的编号、相应边的权值等。此时的空间需求是$O(n+e)$（其中e表示图中包含的边的数目），它对于图的大小而言是线性的。图 6.1a 的无向图的邻接表如图 6.2a 所示；图 6.1c 的有向网的邻接表如图 6.2b 所示。



a) 图 6.1 中无向图的邻接表



b）图 6.1 中有向网的邻接表

图 6.2 图的邻接表表示法

图的邻接表存储结构的类型声明如下：

```
#define MAX _V 20                  // 最大顶点数目
typedef struct ArcNode             // 边的结点结构类型
{
    int adjvex;                    // 该条边的终点编号
    int weight;                    // 该条边的权值（用于网）
    struct ArcNode *nextarc;       // 指向下一条边的指针
} ArcNode;
typedef struct VertexNode          // 邻接表顶点结构
{
    VertexType data;               // 顶点信息
    ArcNode *firstarc;             // 指向第一条与该顶点有关的边的指针
} VertexNode;
typedef struct                     // 图的邻接表结构类型
{
    VertexNode   vnode[MAX_V]; // 定义邻接表
    int vexnum,arcnum;             // 图包含的顶点数与边的个数
    VertexType vexs[MAX_V];  // 存放顶点信息
    GraphKind type;                // 图的种类标志，分为无向图、有向图、无向网和有向网
} ALGraph;
```

以不带权值的图为例，创建以邻接表表示的无向图的算法实现如下。

算法 6.2 基于邻接表表示法的无向图的创建

```
status CreateGraph( ALGraph &G )
{
    scanf( &G.vexnum, &G.arcnum );      // 输入图的基本信息，包括顶点与边的数目
    for( i=0; i<G.vexnum; i++)
        scanf(&G.vexs[i]);              // 输入顶点信息
    ArcNode *p;
    for( i=0; i<G.vexnum; i++ )
        G.vnode[i].firstarc = NULL;     // 邻接表初始化，所有单向链表均为空表
    for( k=0; k<G.arcnum; k++ )
    {
        scanf(&i,&j);                                  // 输入一条边，i、j 表示边的两个顶点的编号
        p = (ArcNode *)malloc(sizeof(ArcNode));  // 创建一个用于存放当前边的结点 p
        p->adjvex = j;                                 // 这条边起始点是 i，终止点是 j
```

```
            p->nextarc = G.vnode[i].firstarc;              // 将这个结点 p 链接到表示 i 号顶点的单链表后
            G.vnode[i].firstarc = p;
            p = (ArcNode *)malloc(sizeof(ArcNode));   // 由于是无向图，因此再创建一个表示对称边的结点 p
            p->adjvex = i;                                 // 这条边起始点是 j，终止点是 i
            p->nextarc = G.vnode[j].firstarc;              // 将这个结点 p 链接到表示 j 号顶点的单链表后
            G.vnode[j].firstarc = p;
        }
        return OK;
    }//CreateGraph
```

图的邻接表表示并不是唯一的。这是因为在每个顶点对应的单链表中，与之相关的各条边的链接次序可以是任意的，这取决于在创建邻接表时读入数据的次序。

借助于邻接表，可以很容易找到任意一个顶点的所有邻接点。但如果要判断任意两个顶点v_i和v_j之间是否存在边，则需要遍历第i或第j个单链表。

对于无向图（或者无向网），为了计算某个顶点v_i的度，只需要遍历第i个单链表，其包含的结点数即为顶点v_i的度。而对于有向图，为了计算v_i的出度，只要遍历第i个单链表，其包含的结点数即为顶点v_i的出度；如果要计算v_i的入度，则需要遍历所有单链表，因为结点 adjvex 域的值为i的结点的总数就是顶点v_i的入度。当然，也可以为有向图再增加一个逆邻接表表示，顶点v_i建立的单链表中，每个结点存放以顶点v_i为弧头的边。这样，计算顶点v_i的入度也只需要遍历第i个单链表即可。

另外，有向图的存储方式还有十字链表（orthogonal list）形式。可以看成是将有向图的邻接表和逆邻接表结合起来得到的一种链表形式。在十字链表中，对应于有向图中每一条弧有一个结点，对应于每个顶点也有一个结点。同样，邻接多重表（adjacency multi-list）是无向图的另一种链式存储结构。邻接多重表的结构和十字链表类似，每一条边用一个结点表示，每一个顶点也用一个结点表示。这两种存储方式的具体表示和操作可见其他参考书目，这里不再赘述。

6.4 图的遍历

从给定图中指定的顶点（起始点）出发，按照某种搜索方法沿着图的边访问图中所有的顶点，使得每个顶点只会被访问一次，这个过程叫做图的遍历。图的遍历方法有两种：一种是深度优先遍历（Depth-First Search, DFS），另一种是广度优先遍历（Breadth-First Search, BFS）。

6.4.1 深度优先遍历

深度优先遍历类似于树的先序遍历。一次**深度优先遍历**的基本过程可以用递归的方式来描述：

（1） 从起始顶点v出发，首先访问顶点v；

（2） 选择一个与顶点v相邻接、且没有被访问过的顶点w作为新的起始点，继续深度优先遍历，直到顶点v的所有邻接点都被访问过。

对于无向图来说，若无向图是连通的，则一次深度优先遍历就可以访问到图中所有的顶点；但如果无向图是非连通的，则一次深度优先遍历只能访问到起始点所在连通分量中的所有顶点，而访问不到其它顶点。因此只有从其它连通分量中选择起始点，继续进行深度优先遍历，才能将图中的所有顶点都访问一遍。

对于有向图来说，若起始顶点到图中其它顶点之间都存在路径，则一次深度优先遍历即可保证将图中的所有顶点都访问一遍；否则同样需要再次选择未被访问过的顶点作为新的起始点继续进行深度优先遍历。

图的顶点之间的关系相对复杂，某个顶点v_i既可以是顶点v_j的邻接点，也可以是其它诸如顶点v_k的邻接点。因此为了保证在遍历中每个顶点只会被访问一次，我们需要借助一个辅助数组 visit[]来做一下标记。若 visit[i]的值为 true，则表示相应的顶点v_i已经被访问过；若 visit[i]的值为 false，则说明顶点v_i还未被访问。

下面以邻接表为例对图进行深度优先遍历。为了保证每个顶点都被访问到，需要对每个未被访问过的顶点调用一次深度优先遍历。

算法 6.3 基于邻接表表示法的图的深度优先遍历

```
void DFS_Visit( ALGraph  G )
{    // 图的深度优先遍历
     for( i=0; i<G.vexnum; i++)
          visit[i] = false;                    // 辅助数组初始化，初始情况下所有顶点均未被访问过
     for( i=0; i<G.vexnum; i++)
     {
          if( visit[i]==false )
               DFS( G, i );                    // 对每一个未被访问过的顶点均调用一次深度优先遍历
     } // end for
}// DFSvisit
```

算法 6.4 以 index 为起始点的深度优先遍历

```
void DFS( ALGraph G, int index )
{
     ArcNode *p;
     visit[index] = true;                     // 将起始点标记为已被访问的状态
     printf( G.vexs[index] );                 // 输出起始点的信息
     p = G->VexNode[index].firstarc;          // 指针 p 指向起始点的第一个邻接点
     while( p )
     {
          // 遍历起始点的所有邻接点，若邻接点已经被访问过，那么 p 继续向后遍历；
          // 否则以当前邻接点为新的起始点递归进行深度优先遍历
          if ( visit[p->adjvex]==false )
               DFS( G, p->adjvex );
          p = p->nextarc;
     } // end while
}// DFS
```

算法 6.4 的时间复杂度是$O(n+e)$，其中n是图中顶点的个数，e是边的个数。如果采用邻接矩阵表示法，则一次深度优先遍历的时间复杂度为$O(n^2)$，因为需要对矩阵进行遍历。

6.4.2 广度优先遍历

广度优先遍历类似于树的层次遍历。一次**广度优先遍历**的基本过程如下：

（1） 从起始顶点v出发，首先访问顶点v；

（2） 依次访问v的**所有**没有被访问过的邻接点$w_1, w_2, \ldots, w_t$；

（3） 按照$w_1, w_2, \ldots, w_t$的次序，访问它们各自所有未被访问过的邻接点；

（4） 以此类推，直到图中所有与起始点v连通的顶点都被访问过为止。

同样，一次广度优先遍历只能保证将与起始点连通的那些顶点访问到，而其他不与起始点连通的顶点是访问不到的。因此，为了保证图中所有顶点都被访问到，需要对每个未被访问的顶点

进行一次广度优先遍历；同时，为了确保每个顶点只会被访问一次，同样需要一个辅助数组 visit[] 来做标记。

还是以邻接表为例，对图进行广度优先遍历。由于广度优先遍历时一种一层一层向外推进的遍历算法，所以需要使用队列进行辅助。

算法 6.5 基于邻接表表示法的图的广度优先遍历

```
void BFS_Visit( ALGraph G )
{    // 广度优先遍历
     for( i=0; i<G.vexnum; i++ )
          visit[i] = false;                  // 对辅助数组初始化，初始情况下所有顶点均未被访问过
     for(i=0;i<G.vexnum;i++)
     {
          if( visit[i]==false )
               BFS( G, i );                  // 对每一个未被访问过的顶点均调用一次广度优先遍历
     } // end for
}// BFSvisit
```

算法 6.6 以 index 为起始点的广度优先遍历

```
void BFS( ALGraph G, int index )
{
     ArcNode *p;
     SeqQueque Q;                            // 采用队列 Q 存放顶点编号
     InitQueue(Q);
     EnQueue(Q, index);                      // 起始点入队
     visit[index] = true;                    // 入队的顶点标记为已访问状态，防止重复入队
     while( ! QueueEmpty(Q) )
     {
          DeQueue(Q, i);                     // 队头元素出队
          printf(G.vexs[i]);                 // 访问队头元素
          p = G->VexNode[i].firstarc;        // 指针 p 指向当前队头顶点的第一个邻接点
          while( p )
          {
               // 遍历当前队头顶点的所有邻接点，若邻接点已经被访问过，那么 p 继续向后遍历；
               // 否则入队，并且标记为已访问状态，防止重复入队
               if( visit[p->adjvex]==false )
               {
                    EnQueue(Q, p->adjvex);
                    visit[p->adjvex] = true;
               }
               p = p->nextarc;
          } // end while (p)
     } // end while ( !QueueEmpty(Q))
}// BFS
```

算法 6.6 的时间复杂度是$O(n+e)$，n是图中顶点的个数，e是边的个数。如果用邻接矩阵表示法，则一次广度优先遍历的时间复杂度为$O(n^2)$，因为需要对整个矩阵进行遍历。

6.4.3 图的遍历算法应用举例

一般涉及不带权的图的最短或者最长路径时可以考虑采用广度优先遍历的算法；而涉及查找所有简单路径、简单回路的问题时可以采用深度优先遍历的算法；涉及无向图的连通性问题时，则两种遍历算法均可考虑采用。以下均以邻接表表示图为例设计算法。

6.4.3.1 无向图的连通性问题

在对无向图进行遍历时，对于连通图，仅需从任意顶点出发，进行一次深度优先或者广度优先搜索，便可以访问到图中所有的顶点；对于非连通图，则需从多个顶点出发进行搜索，而每次在从一个新的起点出发的遍历过程中访问到的顶点序列恰好为其各个连通分量中的顶点集。

例 6.1 求无向图 G 中的连通分量的个数

利用深度优先或者广度优先遍历算法均可，以深度优先遍历算法为例，在最外层调用几次深度优先遍历，就有几个连通分量。只需要将算法 6.3 稍作修改即可得到其实现算法（算法 6.7）。

算法 6.7 计算无向图的连通分量的个数

```
int GetCCNum( ALGraph G )
{    // 求无向图连通分量的个数
     int count = 0;
     for( i=0; i<G.vexnum; i++)
          visit[i] = false;
     for( i=0; i<G.vexnum; i++)
     {
          if( visit[i]==false )
          {
               DFS(G, i);
               count++;        // 调用 DFS 的次数即为图的连通分量的个数
          } // end if
     } // end for
     return count;
}// GetCCNum
```

如果要利用广度优先遍历算法来实现，那么只要对算法 6.5 进行同样的修改即可。

例 6.2 判断无向图 G 的连通性 直接使用算法 6.7 也可以实现该例，如果函数返回值等于 1，说明图是连通的；否则说明图是非连通的。但该算法，需要将图遍历一遍。效率更高的方法是从图中任意顶点出发调用一次 DFS（或者 BFS），然后观察辅助数组 visit[]是否全被标记过，一旦还有未被标记的点，则说明还有顶点未被访问，即图是非连通的；否则就是连通的。

算法 6.8 判断无向图的连通性

```
Status IsConnectedGraph( ALGraph  G )
{    // 判断无向图 G 是否连通
     for( i=0; i<G.vexnum; i++)
          visit[i] = false;
     DFS( G, 0 );                              // 以第一个顶点为起始点（任意顶点均可）调用一次 DFS
     for( i=0; i<G.vexnum; i++)
     {
          if( visit[i]==false )  return FALSE;    // 如果还有顶点未被访问说明图非连通
     }
     return TRUE;                              // 一次 DFS 后，全部顶点均被访问，说明图是连通的
}// IsConnectedGraph
```

例 6.3 u和v是图 G（可以是有向图，也可以是无向图）中的两个顶点，请设计算法判断顶点u到顶点v是否存在路径。

如果两个顶点之间存在路径，则说明这两个顶点是连通的。因此，这个问题依然是连通性问题，只不过是顶点之间的连通性问题，因此仍然可以利用遍历算法来解决。即以u为起始点调用一次 DFS（或者 BFS），然后观察辅助数组 visit[]中对顶点v的标记情况。如果顶点v已经被访问过，说明u到v是连通的；否则u到v就是非连通的，即u到v不存在路径。

算法 6.9 判断顶点之间的连通性

```
Status IsConnected( ALGraph G, int u, int v )
{
    for( i=0; i<G.vexnum; i++ )
        visit[i] = false;
    DFS( G, u );                // 以 u 为起始点调用一次 DFS
    if( visit[v]==false )       // 观察顶点 v 的访问标记
        return FALSE;           // 如果顶点 v 未被访问，则说明 u 到 v 不连通
    else
        return TRUE;            // 如果顶点 v 已被访问，则说明 u 到 v 是连通的
}// IsConnected
```

6.4.3.2 简单路径问题

例 6.4 G 是一个连通的图，设计算法求距离顶点v最远的任意一个顶点。

利用广度优先遍历算法，从顶点v出发进行广度优先遍历，最后一层的顶点距离v最远。在遍历时利用队列逐层暂存各个顶点，队列中的最后一个顶点k一定在最后一层，因此只要将该顶点作为结果即可。

算法 6.10 求图中距离某顶点最远的任意一个顶点

```
int FindMaxDist( ALGraph G, int v)
{
    ArcNode *p;
    int visit[MAX_V];                   // 辅助数组 visit
    SeqQueue Q;                         // 定义队列
    InitQueue(Q);
    for( i=0; i<G.vexnum; i++)
        visit[i]=false;
    EnQueue(Q, v);                      // 起始点入队
    visit[v] = true;                    // 起始点的访问标记置为已访问，防止重复入队
    while( !QueueEmpty(Q) )
    {
        DeQueue(Q, i);                  // 队头元素出队
        p = G->VexNode[i].firstarc;     // 指针 p 指向当前队头的第一个邻接点
        while( p )
        {
            // 遍历当前队头顶点的所有邻接点，若邻接点已经被访问过，那么 p 继续向后遍历；
            // 否则入队，并且标记为已访问状态，防止重复入队
            if( visit[p->adjvex]==false )
            {
                EnQueue(Q, p->adjvex);
                visit[p->adjvex] = true;
             } // end if
             p=p->nextarc;
      } // end while(p)
    } // end while (!QueueEmpty(Q))
    return i;                           // i 是最后出队的顶点，它就是距离顶点 v 最远的一个顶点
}// FindMaxDist
```

例 6.5 设计算法输出图中顶点u到顶点v的全部简单路径。

利用深度优先遍历，在算法 6.4 的 DFS 算法基础上，设计功能函数 FindPath(ALGraph G, int u, int v, int *path, int d)来实现该算法。其中 G 表示图；u 表示路径的起始点编号；v 表示路径的终止顶

点编号；visit[]是进行访问标记的辅助数组，初始值全部为假；path[]是存放路径的数组；d 表示路径的长度，初始值为 0。

算法 6.11 求图中两个顶点之间的全部简单路径

```
void FindPath( ALGraph G, int u, int v, int *path, int d )
{
    ArcNode *p;
    path[d] = u;                                  // 将当前的起始点放入路径中
    visit[u] = true;                              // 设置当前点的访问标记，防止路径中出现重复顶点
    if( u==v )                                    // 如果当前起始点已经等于终止点，则说明当前已经找到一条路径
    {
        for( i=0; i<=d; i++)
            printf("%d ", path[i]);               // 输出路径中的顶点
        printf("\n");                             // 一条路径结束，输出一个换行符
    }
    else                                          // 如果当前起始点不等于终止点，需要继续搜索下去
    {
        p = G->VexNode[u].firstarc;               // 指针 p 指向当前起始点的第一个邻接点
        while( p )
        {
            if( visit[p->adjvex]==false )         // 如果这个顶点未被访问
            {
                FindPath( G,p->adjvex,v,path,d+1 );  // 则以它作为新的起始点递归的调用自己
                visit[p->adjvex] = false;             // 恢复访问标记，使该顶点可重新使用
            }
            p=p->nextarc;
        } // end while (p)
    } // end else
} // FindPath
```

6.5 最小生成树

一个连通图的**生成树**(spanning tree)是一个极小连通子图，它含有图的全部顶点，但只有足以构成一棵树的$n-1$条边。如果在生成树上再添加一条边，则必定会构成一个环，因为这条边使得依附于它的两个顶点之间有了第二条路径。满足此条件的生成树可能有多棵，即生成树并不唯一。

一棵有n个顶点的生成树有且仅有$n-1$条边。如果一个图有n个顶点和小于$n-1$条边，则必然是非连通图；如果图中的边多于$n-1$条，则必然有环存在。但是，有$n-1$条边的图不一定是生成树。

6.5.1 最小生成树的定义

既然图的生成树不是唯一的，对于边带有权值的图（网）来说同样可以有若干生成树，通常把树中边的权值之和叫做树的权。在图的所有生成树中，树的权值最小的生成树称之为**最小生成树**（Minimum Cost Spanning Tree, MST）。当图中存在若干权值相等的边时，最小生成树也不是唯一的。如果我们需要用最少的电线给一所房子设计电路，或者用最短的光纤将若干不同地点的电脑连起来形成一个局域网，就需要用最小生成树来解决类似的问题。

例如，要在n个城市之间建立通信网络，并希望从任意一个城市都可以通过通信网络到达其它$n-1$个城市。我们知道，连通n个城市只需要$n-1$条通信线路，这时自然会考虑一个关键的问题——如何在最节省经费的情况下修建这个通信网络呢？

任意两个城市之间都可以修建一条通信线路，当然相应地需要付出一定的经济或者时间的代价。已知n个城市之间存在$n(n-1)/2$条线路，那么如何在其中选择$n-1$条线路，使得各个城市之间相互连通并且耗费的代价最小呢？如果用无向网来表示这n个城市以及它们之间可能存在的通信线路，则其中无向网的顶点表示城市，边表示城市之间的线路，边的权值代表修建这条线路所耗费的代价（可能是经济代价，也可能是时间等代价），如图 6.3 所示。那么我们的问题就可以转化为在这样的一个无向网中如何找到最小生成树。

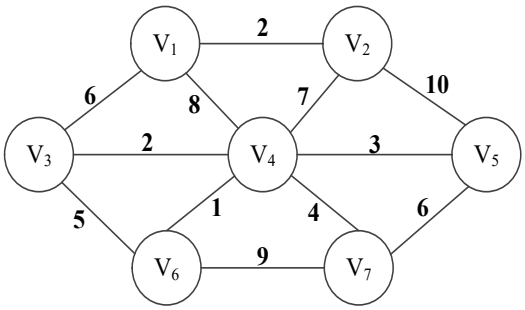


图 6.3 城市公路系统网络示意图

这里我们将介绍两种求最小生成树的算法：普里姆（Prim）算法和克鲁斯卡尔（Kruskal）算法。它们的区别在于看问题的出发点不一样：普里姆算法从顶点的角度出发，逐个将顶点拉入到最小生成树中；而克鲁斯卡尔算法则从边的角度出发，依据权值大小逐个将$n-1$边拉入到最小生成树中。

6.5.2 普里姆算法

在计算最小生成树时，普里姆算法的初始状态仅有一个顶点在最小生成树的顶点集合U中，其他顶点都在另一个由不在最小生成树上的顶点构成的集合V中。在后续的每一步中，通过选择所有连接最小生成树上的顶点u和不在树上的顶点v之间的边中权值最小的边(u,v)，将对应的顶点v拉入最小生成树顶点集合中。当图中所有的顶点都已加入到树中时，算法运行结束，此时得到的n个顶点和$n-1$条边就构成了一棵最小生成树。

图 6.4 给出了普里姆算法如何针对图 6.3 所示的无向网，从v_1顶点开始，一步步构建最小生成树的过程。最初，树的点集中只包含v_1这一个顶点，其后的每一步都添加一个顶点和相应的一条边到树这个点集中，直至所有的顶点都被添加进来，即得到该图的一棵最小生成树。

用V表示原始无向网所包含的顶点集，U表示当前最小生成树中包含的点集。初始情况下，$V=\{v_2,v_3,\dots,v_7\}$，$U=\{v_1\}$。普里姆算法的每一步操作实际就是在点集$V-U$中选取一个顶点，该顶点与U中某个顶点之间的边在点集$V-U$和点集U之间的所有边中权值最小。

该方法的关键在于每次迭代过程中都要选择一条符合上述条件的权值最小边。假设当前集合U中有m个顶点，集合V中有$n-m$个顶点，那么是否每次都要进行$m(n-m)$次遍历寻找权值最小的边呢？显然，可以根据历史比较的结果，以及新的顶点加入到集合U后发生的变化，进行$n-m$次“更新”操作，即可找出新的权值最小的边。这样，算法的时间复杂度就缩小到了原来的$1/m$。

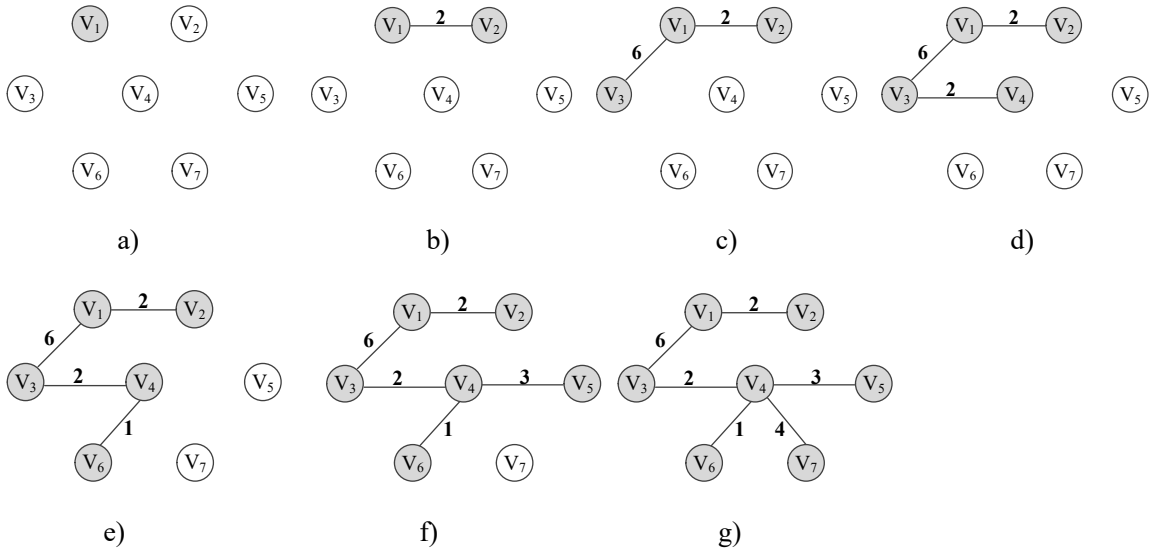


图 6.4 普里姆算法过程示意图

为此，需要一个辅助数组 closest 来记录中间结果。该数组的每个元素都包含两个域，一个是 lowcost 域，表示点集$V-U$与U之间代价最小的边的权值；另一个 vexcode 域表示目前在U中的某个顶点的编号。每个 lowcost 对应一条边，这条边一个顶点在$V-U$中，另一个顶点在U中，如果用 vexcode 表示在U中的顶点，那么另一个在$V-U$中的顶点号就用数组 closest 的位置序号来表示了。图 6.5 显示了 closest 数组的处理变化过程。

	1	2	3	4	5	6	7
vexcode	**1**	1	1	1	1	1	1
lowcost	**0**	2	6	8	∞	∞	∞
tag	**T**	F	F	F	F	F	F

a) closest 数组初始状态

	1	**2**	3	4	5	6	7
vexcode	**1**	**1**	1	1	1	1	1
lowcost	**0**	**2**	6	8	∞	∞	∞
tag	T	T	F	F	F	F	F

b) 选中 2 号顶点 v_2

	1	**2**	**3**	4	5	6	7
vexcode	**1**	**1**	**1**	2	2	1	1
lowcost	**0**	**2**	**6**	7	10	∞	∞
tag	T	T	T	F	F	F	F

c) 数组更新后选中 3 号顶点 v_3

	1	**2**	**3**	**4**	5	6	7
vexcode	**1**	**1**	**1**	**3**	2	3	1
lowcost	**0**	**2**	**6**	**2**	10	5	∞
tag	T	T	F	F	F	F	F

d) 数组更新后选中 4 号顶点 v_4

	1	**2**	**3**	**4**	5	**6**	7
vexcode	**1**	**1**	**1**	**3**	4	**4**	4
lowcost	**0**	**2**	**6**	**2**	3	**1**	4
tag	T	T	T	T	F	T	F

e) 数组更新后选中 6 号顶点 v_6

	1	**2**	**3**	**4**	**5**	**6**	7
vexcode	**1**	**1**	**1**	**3**	**4**	**4**	4
lowcost	**0**	**2**	**6**	**2**	**3**	**1**	4
tag	T	T	T	T	T	T	F

f) 数组更新后选中 5 号顶点 v_5

	1	**2**	**3**	**4**	**5**	**6**	**7**
vexcode	**1**	**1**	**1**	**3**	**4**	**4**	**4**
lowcost	**0**	**2**	**6**	**2**	**3**	**1**	**4**
tag	T	T	T	T	T	T	T

g) 数组更新后选中 7 号顶点 v_7

图 6.5 普里姆算法实现示意图

数组的初始状态如图 6.5a 所示，其中第一行数字表示 closest 数组元素的标号，也用它来表示点集V中的 7 个顶点的编号；第二行数据是 closest 数组的 vexcode 域，初始只选中了第一个顶点v_1进入最小生成树的点集U，因此这一行的数据均为 1；第三行是 closest 数组的 lowcost 域，此时记录

的是 1 号顶点v_1到其余顶点边的权值，由于v_1到v_5、v_6、v_7没有边，因此对应的 lowcost 为极大值；第四行是 closest 数组的 tag 域，记录当前顶点是否已加入点集U，加入后置为 T。由于v_1已经被选中，因此 closest 数组中相应第一个元素的 tag 域置 T。接下来的步骤如下：

（1） 在图 6.5a 的 closest 数组中寻找 tag 域为 F 且 lowcost 域是最小值的顶点。可以发现 2 号元素的 lowcost 最小，因此将 2 号顶点v_2选中进入点集U，如图 6.5b 所示。

（2） 对 closest 数组做更新——首先将 2 号元素的 tag 域置 T ，表示 2 号顶点v_2已被选中；然后观察v_2与剩余未被选中顶点之间边的权值，由于边(v_2, v_4)的权值小于边(v_1, v_4)的权值，(v_2, v_5)的权值小于边(v_1, v_5)的权值，因此更新 closest 数组中的 4 号和 5 号元素的 vexcode 域和 lowcost 域，如图 6.5c 所示。然后在 closest 数组中继续寻找 tag 域为 F 且 lowcost 域取最小值的顶点，此时找到 3 号元素最小，因此将 3 号顶点v_3选中进入点集U，如图 6.5c 所示。

（3） 继续对 closest 数组进行更新——首先将 3 号元素的 tag 域置 T ，表示 3 号顶点v_3已被选中；然后观察v_3与剩余未被选中顶点之间边的权值，由于边(v_3, v_4)和边(v_3, v_6)的权值都比 closest 数组中原先的 4 号和 6 号元素的 lowcost 域小，因此在这两个位置上对 closest 的两个域进行更新，如图 6.5d 所示。然后在当前的 closest 数组中继续寻找 tag 域为 F 且 lowcost 域取最小值的顶点，发现 4 号元素最小，因此将 4 号顶点v_4选中进入点集U，如图 6.5d 所示。

（4） 以此类推，继续对 closest 数组进行更新，然后选出相应的顶点，直到所有的顶点都被选中进入到点集U中，算法结束（如图 6.5e、图 6.5f 和图 6.5g 所示）。

可见，每一轮操作主要有以下 3 个步骤。

第 1 步：在 closest 数组中寻找一个最小权值的边，该边是连接树上顶点 u 和不在树上顶点 k。

第 2 步：通过该最小权值的边，将不在树上的顶点 k“拉入”树上顶点的集合。

第 3 步：由于顶点 k 加入树上顶点集合，比较其他不在树上的顶点 v 到树上顶点的原有最小权值距离和到新的顶点 k 的权值距离，并更新为两者的最小值。

综上，设图以邻接矩阵表示法表示，则普里姆算法实现如下。

算法 6. 12 基于邻接矩阵表示法的普里姆算法

```
struct
{
    int vexcode;
    int lowcost;
    bool    tag;
} closest[MAX_V+1];                          // 定义辅助数组 closest 的结构
void MinSpanningTree_Prim( MGraph G, int v)
{   // 从顶点 v 出发采用 Prim 算法计算图的最小生成树
    for( j=1; j<=G.vexnum; j++ )             // 对数组 closest 进行初始化处理
    {
        closest[j].lowcost  = G.arcs[v][j];
        closest[j].vexcode = v;
        closest[j].tag      = false;
    }
    closest[v].lowcost = 0;                  // 标记 v 已经被选中（第一个被选中的顶点）
    closest[v].tag      = true;
    for( i=2; i<=G.vexnum; i++)              // 将剩余的 G.vexnum-1 个顶点加入到最小生成树中
    {
        min = INFINITY;
        // 第 1 步：在数组 closest 中寻找 lowcost 域的非零最小值
        for( j=1; j<=G.vexnum; j++ )
        {
            if( closest[j].tag==false && closest[j].lowcost<min )
            {
```

```
                min = closest[j].lowcost;
                k = j;                          // 记录具有最小 lowcost 的元素位置
            } // end if
        } // end for j
        // 第 2 步： 选中本轮的最小权值边对应的顶点 k
        printf("边(%d,%d)权为：%d\n", k, closest[k].vexcode, min);
        closest[k].tag=true;                    // 标记 k 已经被选中
        // 第 3 步：由于顶点 k 的加入，对 closest 数组进行更新
        for( j=1; j<=G.vexnum; j++)
        {
            if( closest[j].tag==false && G.arcs[k][j] < closest[j].lowcost )
            {       // 不在树上的顶点到新加入树上顶点 k 的距离比原来的小
                closest[j].lowcost = G.arcs[k][j];
                closest[j].vexcode = k;
            } // end if
        } // end for j
    } // end for i
} // MinSpanningTree_Prim
```

普里姆算法的核心部分是一个双重循环。若无向网中一共有n个顶点，则第一重循环的频度是$n-1$，其中有两个内循环：一个是遍历 closest 数组寻找 lowcost 域的最小值，频度是n；另一个内循环是对 closest 数组进行更新，频度也是n。因此普里姆算法的时间复杂度是$O(n^2)$，与无向网中边的个数没有关系。该算法适合计算稠密图的最小生成树。

6.5.3 克鲁斯卡尔算法

克鲁斯卡尔算法的过程是：连续按照最小的权选择边，当所选边不产生回路时，就把它选定。如果无向网中包含n个顶点，那么需要选定$n-1$条边。对于如图 6.3 所示的无向网，需要选定 6 条边，其实现过程如图 6.6 所示。

如何判定一条边(u,v)能不能被选中呢？即如何判定边(u,v)是否会在当前已选中的最小生成树的点集中形成回路。回路的判断可以使用并查集思想来实现。克鲁斯卡尔算法描述如下。

算法 6.13 基于邻接矩阵表示法的克鲁斯卡尔算法

```
typdef struct edge
{
    int i, j;           // 选中边的两个顶点
    int cost;           // 选中边的权重
} Edge;
void MinSpanningTree_Kruscal( MGraph G, Edge result[] )
{   // 采用 Kruscal 算法计算图的最小生成树
    int  parent[MAX_V+1]; // 用于记录每个顶点对应的集合，便于判断是否构成回路
    Edge  edges[MAX_E+1];   // 存放图 G 中的边信息，从下标 1 开始

    // 1. 将图 G 中的边根据整理到 edges 中并按权重值从小到大排序
    GetEdgesInGraph(G, edges); // 该函数需读者自行补充实现
    // 2. 初始化 parent 数组
    for (i=1; i<=G.vexnum; i++)
    {   parent[i] = i;                  // 每个顶点初始化为跟自身在同一个集合
    }
    // 3. 遍历所有的边
    for (i=1, count=0; i<=G.arcnum; i++)
    {   // 3.1 取得当前边的两个顶点编号
        start = edges[i].i;
```

```
            end  = edges[i].j;
            // 3.2 判断是否会构成回路
            if ( parent[start] == parent[end] ) continue;    // 当前边的加入会构成回路，舍弃
            // 3.3 将当前边选为最小生成树上的边
            result[count].i = edges[i].i;
            result[count].j = edges[i].j;
            result[count].cost = edges[i].cost;
            count++;
            // 3.4 更新 parent 集合
            elem = parent[end];
            for( k=1; k<=G.vexnum; k++)
            {     // 将所有跟顶点 end 在一个集合的顶点并入到跟顶点 start 在同一集合
                  if ( parent[k] == elem)   parent[k] = parent[start];
            }
            if (count == G.vexnum-1 ) break;   // 已找到 n-1 条边
      }
} // end MinSpanningTree_Kruscal
```

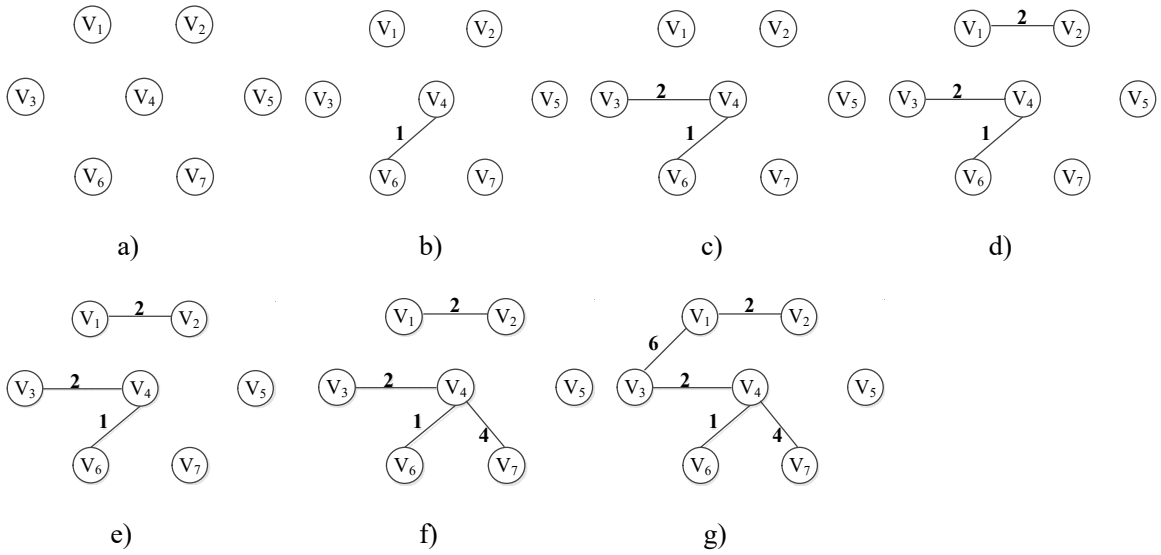


图 6.6 克鲁斯卡尔算法示意图

显然，事先将边按照权值排序会更便于选择，可以考虑利用堆来进行这个排序。克鲁斯卡尔算法的时间复杂度是$O(e\log e)$，其中 e表示无向网中边的数目。可见，克鲁斯卡尔算法的复杂度跟边的数目有关。因此该算法适合计算稀疏图的最小生成树。

6.6 拓扑排序与关键路径

6.6.1 拓扑排序

拓扑排序（topological sort）是对有向无环图的顶点的一种排序。如果存在一条从v_i到v_j的路径，那么在拓扑排序中，v_j一定出现在v_i的后面。图 6.7 展示了某大学的课程设置结构图，其中有向边(v,w)表示课程w的先修课程是v。此外，拓扑排序不必是唯一的，任何合理的排序都是可以的。对于图 6.7 来说，以下两个序列都是它的拓扑序列：$\{C_1,C_2,C_3,C_4,C_5,C_6,C_7\}$和$\{C_1,C_3,C_2,C_5,C_4,C_6,C_7\}$。

如果有向图中有回路，那么该图中不存在拓扑序列。因为对于回路上的两个顶点v和w，在v必须要先于w的同时w又要先于v，从而形成了死锁。

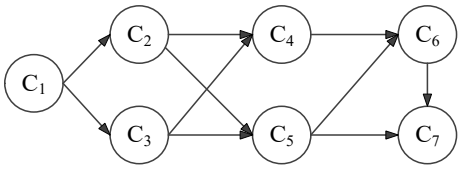


图 6.7 表示课程结构的有向无环图

如何进行拓扑排序呢？一个简单的求拓扑排序的过程如下：

（1）首先在有向图中找到任意一个入度等于 0 的顶点，并将其输出；

（2）然后将该顶点以及与该顶点相关联的边一起从图中删除；

（3）重复上述两步直到图中所有顶点都已被输出，或者图中已不存在入度为 0 的顶点。

在该过程中，找到入度为 0 的顶点是关键。为了与后续的关键路径算法联合，这里用栈结构来存储遍历过程中遇到的入度为 0 的顶点。

以采用邻接表表示法的有向图为例，拓扑排序的算法如下。

算法 6.14 基于邻接表表示法的拓扑排序

```
Status TopologicalSort( ALGraph G )
{
    ArcNode *p;
    int count=0;                          // 统计拓扑排序过程中访问到的顶点总数
    SeqStack  S;                          // 使用一个栈结构辅助拓扑排序
    int indegree[MAX_V];                  // 用来存放每个顶点的入度
    for( i=0; i<G.vexnum; i++ )           // 对存储入度的数组初始化
        indegree[i]=0;
    InitStack(S);
    // 第 1 步：遍历邻接表，统计每个顶点的入度
    for( i=0; i<G.vexnum; i++ )
    {
        p = G.VexNode[i].firstarc;
        while( p )
        {
            indegree[p->adjvex]++;        // 被指向结点的入度加 1
            p=p->nextarc;
        }
    }
    // 第 2 步：将所有入度为 0 的顶点入栈
    for( i=0; i<G.vexnum; i++ )
    {
```

```
        if( indegree[i]==0 )  Push(S, i );
    }
    while( !StackEmpty(S) )                    // 在栈非空时进行循环
    {
        // 第 3 步：通过栈实现拓扑排序的输出
        Pop( S, i );                           // 弹栈
        printf(G.vexs[i]);                     // 输出弹出的顶点
        count++;                               // 记录访问的顶点数目
        // 第 4 步：输出入度为 0 的顶点后，更新与之邻接的顶点的入度
        p = G.VexNode[i].firstarc;
        while( p )                             // 将当前顶点以及相关联的边删除，对入度的数组做更新
        {
            indegree[p->adjvex]--;             // 对应顶点的入度减 1
            if( indegree[p->adjvex] ==0)       // 如果入度为 0 则入栈
                Push(S, p->adjvex );
            p=p->nextarc;
         } // end while p
    } // end while ( !StackEmpty(S) )
    if( count<G.vexnum )
        return FALSE;          // 如果访问到的顶点数小于图的顶点总数，则图中有环，拓扑排序不成功
    else
        return TRUE;           // 否则，拓扑排序成功
}// TopologicalSort
```

上述算法的时间复杂度为$O(n+e)$，n和e分别是有向图中顶点的数量和边的数量。如果利用邻接矩阵表示法，则拓扑排序的时间复杂度为$O(n^2)$。上面我们提到有向图中如果有回路，则不存在拓扑序列，因此也可以利用拓扑排序来判断一个有向图是否有环，即如果拓扑排序成功，则说明有向图中没有环，否则说明有向图中有环。

6.6.2 AOE 网与关键路径

有向无环网的一个重要应用是关键路径分析。

若在有向网中，以顶点表示事件；边代表具有优先关系的活动，即一条有向边(v,w)表示该活动中事件v必须发生在事件w之前；有向边(v,w)的权值表示活动需要花费的时间或者代价。当然，这意味着有向网必须是无环的。这种类型的图经常用来模拟工程方案的构建，也称为 AOE（Activity On Edge）网。在这种情况下，有几个重要的问题需要解答：首先，工程的最早完成时间是何时？其次，哪些活动是影响整个工程进度的关键？

由于一般的工程都只有一个开始点和一个完成点，所以正常情况下，一个表示工程方案的 AOE 网中（无环）只有一个入度为零的顶点，称为**源点**，表示工程方案的开始，以及一个出度为零的点，称为**汇点**，表示工程方案的结束。在 AOE 网中，从源点到汇点最长的路径（这里的路径长度是路径中包含的边的权值之和，并不是路径中包含的边的数目）**称为关键路径**，完成工程需要的最短时间就是关键路径的长度。

假设 AOE 网中，x是源点，z为汇点，下面给出在求取关键路径时需要用到的一些变量与概念。

（1） 事件k的**最早开始时间**$ve(k)$，是从源点到顶点k的最长路径长度。事件的最早开始时间决定了所有从事件k开始的活动能够开工的最早时间。推算公式为

$$ve(x)=0$$

$$ve(k)=MAX\{ve(j)+w(j,k)|(j,k)\in T\} \qquad (6.1)$$

其中$w(j,k)$表示边(j,k)的权值，T表示所有终止点为k的边的集合。

（2） 事件k的**最晚开始时间**$vl(k)$，是在不推迟整个工期的前提下，事件k最迟必须开始的时间。它等于整个工程的关键路径的长度减去顶点k到汇点的最长路径长度。推算公式为

$$vl(z) = ve(z)$$
$$vl(k) = MIN\{vl(j) - w(k,j) | (k,j) \in P\} \quad (6.2)$$

其中$w(k,j)$表示边(k,j)的权值，P表示所有起始点为k的边的集合。

（3） 事件k的最晚开始时间与最早开始时间之差，意味着事件k的开始有一个时间余量。若$vl(k) = ve(k)$，则k为一个关键事件。

（4） 若一个活动(v,w)，其起始点v和终止点w均为关键事件，则该活动即为**关键活动**。显然，关键路径上的活动都是关键活动，提前完成非关键活动并不能加快工程的进度。

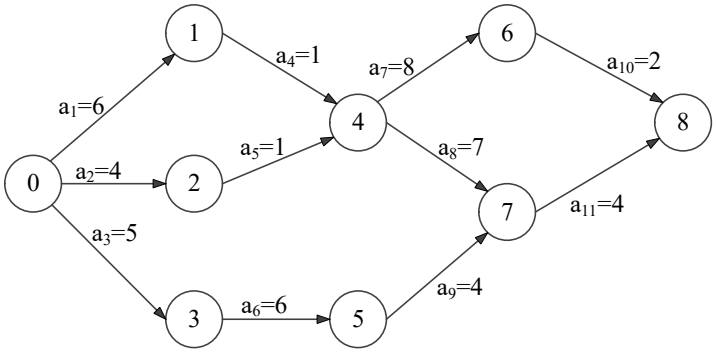


图 6.8 一个 AOE 网

上述两个递推公式（式（6.1）和式（6.2））的计算必须在拓扑有序和拓扑逆有序的前提下进行。也就是说事件k的最早开始时间$ve(k)$必须在k的所有前驱的最早开始时间计算完成后才能确定；而事件k的最晚开始时间$vl(k)$则必须在 k 的所有后继的最迟开始时间计算完成后才能确定。因此，应该在拓扑排序的基础上计算每个事件的最早和最迟开始时间。

以图 6.8 所示的 AOE 网为例，图中包含 11 个活动、9 个事件。顶点 0 是源点，顶点 8 是汇点，边的权值代表活动所需时间。其拓扑序列为：{0,3,5,2,1,4,6,7,8}。首先计算各个顶点的最早开始时间：

(1) $ve(0) = 0$
(2) $ve(3) = 5$
(3) $ve(5) = ve(3) + 6 = 11$
(4) $ve(2) = 4$
(5) $ve(1) = 6$
(6) $ve(4) = MAX(ve(1) + 1, ve(2) + 1\} = 7$
(7) $ve(6) = ve(4) + 8 = 15$
(8) $ve(7) = MAX(ve(4) + 7, ve(5) + 4\} = 15$
(9) $ve(8) = MAX(ve(6) + 2, ve(7) + 4\} = 19$

然后求所有事件的最迟开始时间：

(1) $vl(8) = ve(8) = 19$
(2) $vl(7) = vl(8) - 4 = 15$
(3) $vl(6) = vl(8) - 2 = 17$
(4) $vl(4) = MIN(vl(6) - 8, vl(7) - 7\} = 8$
(5) $vl(1) = vl(4) - 1 = 7$
(6) $vl(2) = vl(4) - 1 = 7$
(7) $vl(5) = vl(7) - 4 = 11$
(8) $vl(3) = vl(5) - 6 = 5$
(9) $vl(0) = MIN(vl(1) - 6, vl(2) - 4, vl(3) - 5\} = 0$

基于此，可以得到每个顶点（事件）的最早开始时间和最迟开始时间列表如下。

表 6.1 事件的最早开始时间和最迟开始时间

事件	**0**	1	2	**3**	4	**5**	6	**7**	**8**
ve	**0**	6	4	**5**	7	**11**	15	**15**	**19**
vl	**0**	7	7	**5**	8	**11**	17	**15**	**19**

由此可以发现，事件 0、3、5、7 和 8 都是关键事件，由这些事件所决定的活动a_3、a_6、a_9、a_{11}都是关键活动。这些关键活动构成了关键路径。因此，分析关键事件、关键路径的目的是辨别哪些是关键活动，以便控制整个工期进度。注意，一个 AOE 网中可能存在多条关键路径。当然，缩短关键活动的时间，不一定会直接缩短整个工期的相应时间，因为这有可能其他的路径成为新的关键路径。

根据上述计算规则，可以较为方便地实现关键路径求解的算法。需要注意的是，计算最早开始时间和拓扑排序的顺序一致，两者可同时完成。最晚开始时间的计算过程恰好是拓扑排序的逆序。所以，在拓扑排序计算过程中，可以在顶点弹栈输出后立即压入另外一个栈，形成拓扑排序的逆序，然后根据这个逆序进行最晚开始时间的计算。具体算法如下。

算法 6.15 拓扑排序并求解各顶点最早开始时间

```
Status TopologicalOrder( ALGraph G, SeqStack &T )
{   // 有向网 G 采用邻接表存储，求解各顶点最早开始时间 ve
    // T 为拓扑序列顶点栈，S 为零入度顶点栈
    FindInDegree(G, indegree);   // 对各顶点求入度
    InitStack( S );
    PutZeroInDegreeInStack( S, indegree );              // 将零入度顶点压入栈 S
    InitStack(T); count = 0; ve[0..G.vexnum-1] = 0;   // 初始化
    while( !StackEmpty(S))
    {     Pop(S, j); Push(T, j); count++;                  // 从 S 栈移到 T 栈中
          for ( p = G.VexNode[j].firstarc; p; p = p->nextarc)
          {     k = p->adjvex;     indegree[k]--;          // 顶点 k 入度减 1
                if ( indegree[k] == 0 ) Push(S, k);       // 入度为零，压入栈 S
                if ( ve[j]+ p->weight > ve[k] )           // 更新 ve
                {   ve[k] = ve[j] + p->weight;   }
          } // end for p
    } // end while
    if ( count < G.vexnum ) return ERROR;    // 图 G 存在回路
    else return OK;
} // TopologicalOrder
```

算法 6.16 关键路径算法

```
Status CriticalPath( ALGraph G )
{   // 求解图 G 的关键路径
    // 1. 拓扑排序并求解 ve
    if ( ! TopologicalOrder(G, T) ) return ERROR;
    // 2. 求解 vl
    vl[0..G.vexnum-1] = max(ve);  // 将汇点的 ve 值作为初始值赋给所有顶点的 vl
    while( !StackEmpty(T) )       // 按拓扑逆序求各顶点 vl
    {     for ( Pop(T, j), p = G.VexNode[j].firstarc; p; p = p->nextarc)
          {     k = p->adjvex;
                if  ( vl[k] - p->weight < vl[j] )  vl[j] = vl[k] - p->weight;   // 更新 vl[j]
          } // end for
     } // end while
    for( j=0; j<G.vexnum; j++)   // 求 ee, el 和关键活动
    {     for( p=G.VexNode[j].firstarc; p; p=p->nextarc )
          {     k = p->adjvex;
                ee = ve[j];   el = vl[k] – p->weight;
                tag = (ee==el) ? '*' : ' ';
                printf ( j, k, p->weight, ee, el, tag);   // 输出当前边是否为关键活动
          }
```

```
            }
        } // CriticalPath
```

6.7 最短路径问题

如图 6.9 所示，假若以顶点表示城市，边表示城市之间的交通联系，边的权值代表相应城市之间交通联系的代价，这个代价可以是城市之间的距离、途中需要的时间或者交通费用等。那么，利用这个图可以回答出行旅客关于路线的一些咨询。例如，旅客想知道从城市 A 到城市 B 的哪条交通线路的中间换乘次数最少。假如每个顶点都表示一次换乘，那么这个问题其实就是从顶点 A 到顶点 B 的路径中寻找一条包含边的数量最少的路径。这个问题可以利用我们前边介绍的图的广度优先遍历算法完成。但是，更多时候旅客可能更关心交通代价的问题，比如要花多少钱、路上要用多少时间，而司机会更关心交通线路的距离问题。此时，路径的长度不应该用路径中边的数量来衡量，而要用路径中包含的边的权值之和来表示。考虑到交通的有向性（比如单行线，上坡或者下坡，水运中的逆水和顺水等），本节将讨论有向网的**最短路径问题**。

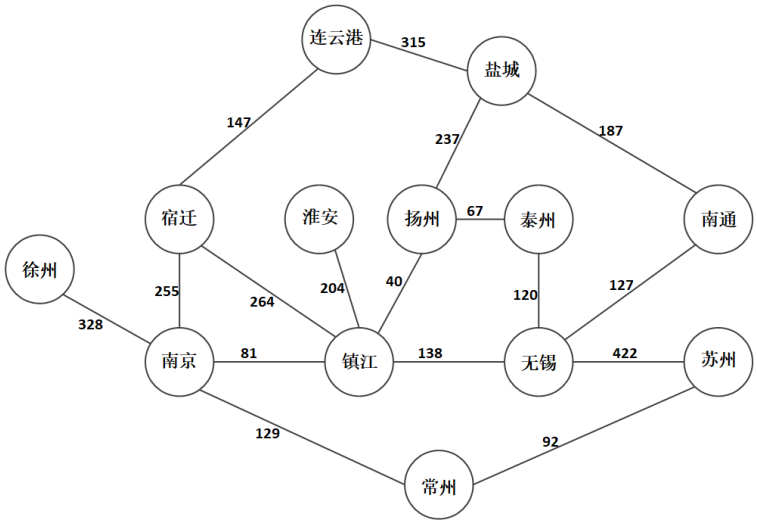


图 6.9 一个表示交通网的示例图

6.7.1 单源最短路径问题

首先讨论单源点的最短路径问题，即在有向网中指定一个起始点v，找出从v到有向网中其余每个顶点的最短路径。解决单源最短路径问题的一个经典方法叫作**迪杰斯特拉**（Dijkstra）**算法**。该算法按阶段进行，在每个阶段，算法选择一个顶点u，该顶点在所有剩余顶点中与起始点v的距离最近，然后以此距离为基础，更新起始点v与其余未被选中的顶点之间的路径距离，接着继续进行选择，直到所有顶点都被选择到，算法结束。

求图G中从顶点v到其余顶点最短路径的迪杰斯特拉算法的具体步骤如下：

（1） 假设S为已求得最短路径的终点的点集，初始时，S中只包含起始点v，而点集U包含图中除了顶点v以外的所有顶点。点集S与U的关系是$S \cap U = \Phi$，且$S + U = G$

（2） 从U中选择一个顶点u，它是源点v到U距离最近的一个顶点，将u加入到S中。

（3） 继续从U中选择下一个与源点距离最近的顶点。该如何选择呢？可以证明：下一条最短路径或者是一个具有最小权值的边(v,w)，其中$w \in U$，或者是中间只能经过S中的顶点而到达终点w（$w \in U$）的路径。将选中的顶点w加入点集S中。

（4） 重复步骤 3，直到点集U为空。

以图 6.10 所示的有向网为例，假设以顶点 0 为起始点，求它到其余顶点的最短距离。

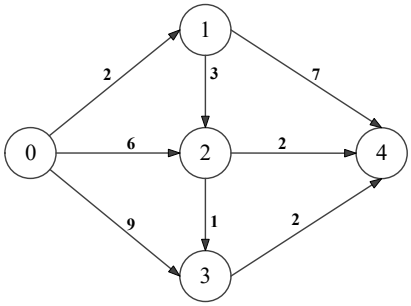


图 6.10 有向网（Dijkstra 算法示例）

为了实现上述的迪杰斯特拉算法，我们需要三个辅助数组，dist[]、path[]和 flag[]。其中 dist[i]表示 0 号顶点到顶点 i 的最短距离；若 path[i]=j，表示从 0 号顶点到达顶点 i 会经过顶点 j；flag[i]表示顶点 i 的选中状态，TRUE（T）表示已被选入点集S，否则表示未被选中。图 6.11 展示了在计算最短路径过程中，这三个数组的取值情况。

（1） 在初始情况下，只有起始点即顶点 0 被选中，其余顶点均是未选中状态。dist 数组中记录的是 0 号顶点与其余顶点之间当前的最短路径，即起始点与其余顶点边的权值，如果不存在边，则权值为无穷大。

（2） 在当前的 dist 数组中选择一个最小值，且相应的 flag 标志为“F”。如图 6.11b 所示，顶点 1 被选中，将其 flag 标志设为“T”；然后观察如果以顶点 1 为中转，则起始点到其余顶点的最短距离是否会减少。由于 1 号顶点到 2 号和到 4 号顶点都存在有向边，因此，如果以 0 号顶点出发经过 1 号顶点到达 2 号顶点，则路径长度为 5。这个长度比原先的从顶点 0 直接到顶点 2 的路径长度 6 要小，因此更新 dist[2]=5，更新 path[2]=1。同理，如果以 0 号顶点出发经过 1 号顶点到达 4 号顶点，则路径长度为 9，该长度比原先的无穷大要小，因此更新 dist[4]=9，更新 path[4]=1。

（3） 继续在当前更新的数据中寻找一个 flag 标志为“F”同时 dist 最小的值。如图 6.11c 所示，顶点 2 被选中。然后按照同样的方式在选中新的顶点 2 后更新数据表。

（4） 重复上述步骤，直到所有顶点均被选中，如图 6.11e 所示，算法结束。此时，从顶点 0 到其余 4 个顶点的最短路径保存在数组 path 中，而相应的路径长度则保存在数组 dist 中。最后得到的结果如下：

顶点 0 到顶点 1 的最短距离为 2，最短路径为 $0 \rightarrow 1$。

顶点 0 到顶点 2 的最短距离为 5，最短路径为 $0 \rightarrow 1 \rightarrow 2$。

顶点 0 到顶点 3 的最短距离为 6，最短路径为 $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$。

顶点 0 到顶点 4 的最短距离为 7，最短路径为 $0 \rightarrow 1 \rightarrow 2 \rightarrow 4$。

	0	1	2	3	4
dist	0	2	6	9	∞
path	0	0	0	0	-
flag	T	F	F	F	F

a)初始情况

	0	1	2	3	4
dist	0	2	5	9	9
path	0	0	1	0	1
flag	T	T	F	F	F

b)第一次选中顶点 1，并更新数据

	0	1	2	3	4
dist	0	2	5	6	7
path	0	0	1	2	2
flag	T	T	T	F	F

c)第二次选中顶点 2，并更新数据

	0	1	2	3	4
dist	0	2	5	6	7
path	0	0	1	2	2
flag	T	T	T	T	F

d) 第三次选中顶点 3，并更新数据

	0	1	2	3	4
dist	0	2	5	6	7
path	0	0	1	2	2
flag	T	T	T	T	T

e)第四次选中顶点 4，算法结束

图 6.11 迪杰斯特拉算法实现示意图

以邻接矩阵表示有向网为例，迪杰斯特拉算法的实现如下。

算法 6.17 基于邻接矩阵表示法的迪杰斯特拉算法

```
void Dijkstra(MGraph G,int s)                    // s 表示起始点编号
{
    int dist[MAX_V],path[MAX_V];                 // 设置三个辅助数组
    bool flag[MAX_V];
    for( i=0; i<G.vexnum ;i++)                   // 对辅助数组做初始化
    {
        flag[i] = false;
        dist[i] = G.arcs[s][i];
        if( dist[i]<INFINITY )     path[i] = s;
        else                       path[i] = -1;
    }
    flag[s] = true;                              // 将起始点设置为被选中状态
    for( i=0; i<G.vexnum-1; i++ )                // 循环选中剩余的顶点
    {  // 第 1 步：寻找当前轮中离源点最近的顶点 k
       k = MIN(dist[ ]);                         // 在 dist 数组中寻找 flag 标志为 false 的元素的最小值
       flag[k] = true;                           // 找到后将其设置为选中状态
       mindist = dist[k];                        // 将找到的最小值赋值给一个变量
       if( mindist<INFINITY )
       {   // 第 2 步：根据本轮选中的顶点 k，更新其他顶点到源点的距离
           for( j=0; j<G.vexnum; j++)
           {    // 顶点 j 经过 k 到源点比原来更近
                if( flag[j]==false && dist[j]>mindist+G.arcs[k][j] )
                 {
                      dist[j] = mindist+G.arcs[k][j];
                      path[j] = k;
                 } // end if
            } // end for
       } // end if
    } // end for
}// Dijkstra
```

由上述算法可以发现，迪杰斯特拉算法的时间复杂度是$O(n^2)$， n表示图中顶点的个数。无论是用邻接矩阵表示法，还是用邻接表表示法，辅助数组 dist[]的长度都与顶点数目相关，需要遍历$n-1$次的辅助数组。因此无论是哪种方式表示图，该算法的时间复杂度均为$O(n^2)$。

如何将 path 数组中保存的最短路径输出留给读者自行完成。

6.7.2 所有顶点对之间的最短路径

有时候需要找出图中所有顶点对之间的最短路径。虽然可以通过调用n次的迪杰斯特拉算法来实现。但是一种更为高效的做法是采用**弗洛伊德**（Floyd）**算法**。若对于有向网G，用一个二维数组D存放两两顶点之间的最短路径长度，即$D[i][j]$表示当前顶点i到顶点j的最短距离。弗洛伊德算

法的基本思想是产生一个矩阵序列D_0、D_1、…、D_k、…、D_{n-1}，其中$D_k[i][j]$表示从顶点i到顶点j的路径上所经过顶点编号不大于k的最短路径长度。

初始情况下，若有向网G以邻接矩阵表示，则$D_{-1}[i][j] = G.arcs[i][j]$。若$D_{k-1}[i][j]$已经求出，接下来要求$D_k[i][j]$时，则此时从顶点$i$到顶点$j$的最短路径有两种情况：

（1） 一种情况是从顶点i到顶点j的最短路径不会经过编号为k的顶点，此时不需要做调整，$D_k[i][j] = D_{k-1}[i][j]$。

（2） 另一种情况是从顶点i到顶点j的最短路径会经过编号为k的顶点，此时路径被分成两段：一段从顶点i到k，最短距离为$D_{k-1}[i][k]$；另一段从顶点k到j，最短距离为 $D_{k-1}[k][j]$。如果$D_{k-1}[i][k] + D_{k-1}[k][j] < D_{k-1}[i][j]$，则将经过顶点$k$的路径作为新的从顶点$i$到顶点$j$的最短路径。

归纳起来，弗洛伊德算法的基本思想可用如下的表达式来描述：

$$D_{-1}[i][j] = G.arcs[i][j]$$

$$D_k[i][j] = MIN\{D_{k-1}[i][j], D_{k-1}[i][k] + D_{k-1}[k][j]\} \qquad (6.3)$$

该式是一个迭代表达式，D_{k-1}是已经考虑到顶点 0、1、2、…、$k-1$这k个顶点后得到的两两顶点之间的最短路径长度，$D_{k-1}[i][j]$就是在考虑了顶点 0、1、2、…、$k-1$这k个顶点后得到的从顶点i到顶点j的最短路径长度。在此基础上，再考虑顶点k，求出在考虑了 0、1、2、…、k这些顶点后得到的两两顶点之间的最短距离D_k。每迭代一次，从顶点i到顶点j的最短路径上就多考虑一个顶点，这样经过n次迭代后得到的D_{n-1}就是考虑了所有顶点后，两两顶点之间的最短路径长度，也就是最后的解。

另外，再用一个二维数组$path$保存最短路径。若$path[i][j] = k$，表示顶点i到顶点j需要经过顶点k。与迪杰斯特拉算法中采用的方法类似，在算法结束时，对二维数组$path$的值进行追溯，可以得到从顶点i到顶点j的最短路径，不过一般需要用递归的方式来进行追溯。

以图 6.12 所示的有向网为例，其对应的邻接矩阵是

$$\begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & \infty & 0 \end{bmatrix}$$

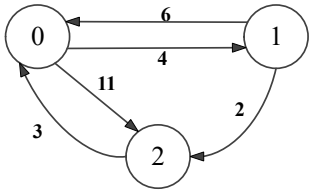


图 6.12 有向网（Floyd 算法示例）

利用弗洛伊德算法的求解过程如下：

（1） 初始有$D_{-1} = \begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & \infty & 0 \end{bmatrix}$，$path_{-1} = \begin{bmatrix} -1 & 0 & 0 \\ 1 & -1 & 1 \\ 2 & -1 & -1 \end{bmatrix}$。

（2） 考虑顶点 0，$D_0[i][j]$表示从i到j经由顶点 0 的最短路径长度。只有顶点 2 经由顶点 0 后到达顶点 1 的路径长度会比$D_{-1}[2][1]$变短，因此有：

$$D_0 = \begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}, \quad path_0 = \begin{bmatrix} -1 & 0 & 0 \\ 1 & -1 & 1 \\ 2 & 0 & -1 \end{bmatrix}。$$

（3） 考虑顶点 1，$D_1[i][j]$表示从i到j经由顶点 1 的最短路径长度。只有顶点 0 经由顶点 1 到达顶点 2 的路径长度会比$D_0[0][2]$变短，因此有：

$$D_1=\begin{bmatrix}0&4&6\\6&0&2\\3&7&0\end{bmatrix},\ path_1=\begin{bmatrix}-1&0&1\\1&-1&1\\2&0&-1\end{bmatrix}。$$

（4） 考虑顶点 2，$D_2[i][j]$表示从i到j经由顶点 2 的最短路径长度。只有顶点 1 经由顶点 2 到达顶点 0 的路径长度会比$D_1[1][0]$变短，因此有：

$$D_2=\begin{bmatrix}0&4&6\\5&0&2\\3&7&0\end{bmatrix},\ path_2=\begin{bmatrix}-1&0&1\\2&-1&1\\2&0&-1\end{bmatrix}。$$

最后求得的各个顶点之间的最短路径长度就保存在最后的D_2中；路径保存在$path_2$中。

以邻接矩阵表示有向网为例，弗洛伊德算法的实现如下。

算法 6.18 基于邻接矩阵表示法的弗洛伊德算法

```
void Floyd( MGraph G )
{
    int D[MAX_V][MAX_V],path[MAX_V][MAX_V];   // 辅助数组
    for( i=0; i<G.vexnum; i++ )                // 对辅助数组初始化
        for( j=0; j<G.vexnum; j++ )
        {
            D[i][j] = G.arcs[i][j];
            if( D[i][j]<INFINITY )  path[i][j] = i;
            else                    path[i][j] = -1;
        }

    for( k=0; k<G.vexnum; k++ )                // 计算 Dk[i][j]
    {
        for( i=0; i<G.vexnum; i++ )
        {
            for( j=0; j<G.vexnum; j++ )
              {
                if( D[i][j] > D[i][k]+D[k][j] )        // 更新
                {
                    D[i][j]    = D[i][k]+D[k][j];
                    path[i][j] = k;
                } // end if
              } // end for j
            }  // end for i
        }  // end for k
    }// Floyd
```

由上述算法可见，弗洛伊德的算法需要通过一个三重循环来实现，因此它的时间复杂度是$O(n^3)$。

利用$path$数组将所有顶点对之间的最短路径进行输出显示留给读者自行完成。

6.8 小结

本章主要讨论了图的抽象数据类型、图的顺序和链式存储方式，以及图的深度遍历、广度遍历和遍历的应用举例，还讨论了图的最小生成树（普里姆算法和克鲁斯卡尔算法）、图的拓扑排序和关键路径、图的最短路径（迪杰斯特拉算法和弗洛伊德算法）等图的应用。图与树不同的是，图的数据元素不仅是顶点本身的信息，还包括边的信息。

特别地，需注意分析和理解各类图的应用过程中，每轮迭代更新策略的思想和方法。最小生成树、关键路径和最短路径算法中都包含迭代更新处理，但这三者的更新策略各有特点。

另外，需要从应用本身出发，注意顶点和边的特点，从而选择合理的表示和存储方式，以及相应的算法策略，进而改善算法的时间复杂度。例如，在计算最小生成树时，对于稠密图，可采用普里姆算法，从顶点着手进行计算；而对于稀疏图，则可采用克鲁斯卡尔算法，从边着手进行计算。

6.9 练习

1. 具有 n 个顶点且每一对不同的顶点之间都有一条边的无向图被称为（）
   A. 无向完全图　　B. 无向连通图
   C. 无向强连通图　　D. 无向树图

2. 一个有 n 个顶点的无向图最多有（）条边。
   A. n　　B. n(n-1)
   C. n(n-1)/2　　D. 2n

3. 设 G 是一个非连通无向图，有 15 条边，则该图至少有（）个顶点。
   A. 5　　B. 6
   C. 7　　D. 8

4. 对于具有 n (n>1)个顶点的有向强连通图，其有向边的条数至少是（）.
   A. n+1　　B. n
   C. n-1　　D. n-2

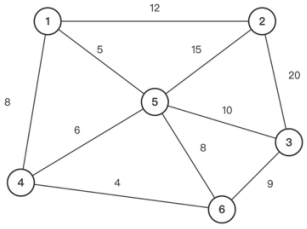
5. 图的广度遍历时，采用（）结构进行辅助。
   A. 栈　　B. 队列
   C. 二维数组　　D. 以上都不是

6. 有 n 个顶点和 e 条边的无向图采用邻接矩阵存储，零元素个数为（）
   A. e　　B. $2e$
   C. $n^2 - e$　　D. $n^2 - 2e$

7. 一个连通图的生成树是含有该连通图全部顶点的（）
   A. 极小连通子图　　B. 极小子图
   C. 极大连通子图　　D. 极大子图

8. 任何一个连通图的最小生成树（）。
   A. 只有一棵　　B. 有一棵或多棵
   C. 一定有多棵　　D. 可能不存在

9. 已知一个带权连通图如下图所示，该图的最小生成树权值之和为（）。



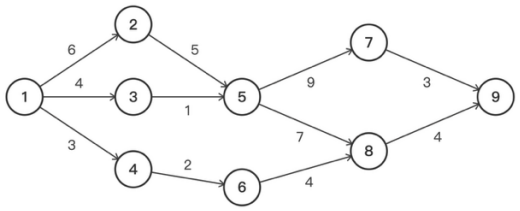
A. 31　　B. 38
C. 36　　D. 43

10. 以下关于最小生成树说法正确的是（）.
A. 最小生成树是指边数最少的生成树
B. 从 n 个顶点的连通图中选取 n-1 条权值最小的边，即可构成最小生成树。
C. 只要带权无向图没有权值相同的边，其最小生成树就唯一。
D. 只要带权无向图中有权值相同的边，其最小生成树就不可能是唯一的。

11. 在用 Kruskal 算法求解带权连通图的最小生成树时，通常采用一个（）辅助结构，判断一条边的两个端点是否在同一个连通分量上。
A. 位向量　　B. 堆
C. 并查集　　D. 生成树顶点集合

12. 以下有关拓扑排序的说法错误的是（）。
A. 拓扑排序成功仅限于有向无环图。
B. 任何有向无环图的顶点都可以排到拓扑排序序列中，而且拓扑序列不唯一。
C. 在拓扑排序序列中任意两个相继排列的顶点 Vi 和 Vj 在有向无环图中都存在从 Vi 到 Vj 的路径。
D. 若有向图的邻接矩阵中对角线以下元素均为零，则该图的拓扑序列必定存在。

13. 下图中，关键路径长度为（）。



A. 23　　B. 22
C. 16　　D. 13

14. 下列有关关键路径的说法错误的是（）。
A. 在 AOE 网络中可能存在多条关键路径
B. 关键活动不按期完成就会影响整个工程的完成时间
C. 任何一个关键活动提前完成，那么整个工程将会提前完成
D. 所有关键活动提前完成，那么整个工程将可能提前完成

15. 若一个有向图中的部分顶点不能通过拓扑排序排到一个拓扑有序的序列里，则可断定该有向图是个（）
A. 有根有向图　　B. 强连通图
C. 含有多个入度为 0 的顶点的图　　D. 含有顶点个数大于 1 的强连通分量

16. 设有向图具有 n 个顶点和 e 条边，如果用邻接表做存储结构，则拓扑排序的时间复杂度为（）
A. $O(n)$　　B. $O(n+e)$
C. $O(n^2)$　　D. $O(n \times e)$

17. 设有向图具有 n 个顶点和 e 条边，如果用邻接矩阵做存储结构，则拓扑排序的时间复杂度为（）

A. $O(e^2)$　　B. $O(n+e)$

C. $O(n^2)$　　D. $O(n \times e)$

18. 下列有关最短路径的说法正确的是（）。

A. 带权有向图的最短路径一定是简单路径.

B. 在有向图中，从一个顶点到另一个顶点的最短路径是唯一的

C. 求单源最短路径的 Dijkstra 算法不适用于有回路的带权有向图。

19. 求最短路径的 Floyd 算法的时间复杂度为（）

A. $O(n)$　　B. $O(n*e)$

C. $O(n^2)$　　D. $O(n^3)$

20. 求最短路径的 Dijkstra 算法的时间复杂度为（）。

A. $O(n)$　　B. $O(n+e)$

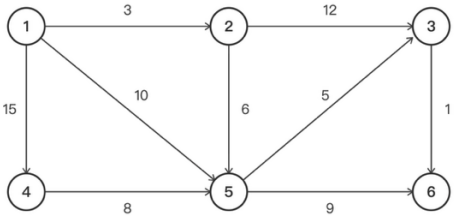
C. $O(n^2)$　　D. $O(n \times e)$

21. 当各边上的权值（）时，可以使用 BFS 算法来解决单源最短路径问题。

A. 都相等　　B. 都不相等

C. 不一定都相等　　D. 都大于 0

22. 如下图所示，依据 Dijkstra 算法求从顶点 1 到其余各顶点的最短路径的顶点顺序是（）



A. 2，5，4，6，3　　B. 2，5，3，4，6

C. 2，3，5，4，6　　D. 5，4，6，3，2

23. Dijkstra 算法是按（）方法求出图中从某顶点到其余顶点最短路径的。

A. 长度递减的顺序　　B. 长度递增的顺序

C. 通过深度优先遍历　　D. 通过广度优先遍历

24. 编程实现图的深度优先和广度优先遍历，测试数据图的规模为至少 100 个顶点和 200 条边。

25. 编程实现 Prim、Kruskal 算法，测试数据图的规模为至少 50 个顶点和 100 条边。

26. 编程实现关键路径求解，测试数据图的规模为至少 50 个顶点和 100 条边。

27. 编程实现 Dijkstra、Floyd 算法，测试数据图的规模为至少 50 个顶点和 100 条边。

28. 对于一个无向图G，给定某个顶点v，如何求解所有以v为起始点（v也是终点）的简单回路？请编程实现。

29. 在无向图G中，请输出从顶点v到顶点u所有长度为k的简单路径。

30. 如果不采用拓扑排序，如何判断一个有向图中是否有回路？

31. 在有向图G中，若从顶点v到图中所有其它顶点都存在路径，则顶点v就是一个根结点。请编程输出有向图G中的所有根结点。

32. 在一个无向网中，以顶点v为起始点，调用一次迪杰斯特拉算法可以得到从v到其余所有顶点的最短路径。这些最短路径与图中的顶点是否可以构成一棵树？如果可以，这棵树是否是一个最小生成树？请说明原因。

33. 调用N次迪杰斯特拉算法和一次弗洛伊德算法的时间复杂度都是$O(N^3)$，它们的执行效率也一样吗？请说明理由（其中N表示图中顶点的数目）。

34. 社交网络图中结点的“重要性”计算

[问题描述]

在社交网络中，个人或单位（结点）之间通过某些关系（边）联系起来。他们受到这些关系的影响，这种影响可以理解为网络中相互连接的结点之间蔓延的一种相互作用，可以增强也可以减弱。而结点根据其所处的位置不同，在网络中体现的重要性也不尽相同。

“紧密度中心性”是用来衡量一个结点达到其他结点的“快慢”的指标，即一个有较高中心性的结点比有较低中心性的结点能够更快地（平均意义下）到达网络中的其他结点，因而在该网络的传播过程中有更重要的价值。在有N个结点的网络中，结点v_i的“紧密度中心性”$Cc(v_i)$数学上定义为　到其余所有结点$v_j(j \neq i)$的最短距离$d(v_i, v_j)$的平均值的倒数：

$$Cc(v_i) = \left[\frac{1}{N-1}\sum_{i\neq j}^{N} d(v_i, v_j)\right]^{-1} = \frac{N-1}{\sum_{i\neq j}^{N} d(v_i, v_j)}$$

对于非连通图，所有结点的紧密度中心性都是 0。

本实验给定一个无权的无向图以及其中的一组结点，要求计算这组结点中每个结点的紧密度中心性。

[基本要求]

（1）输入说明：输入的第一行给出两个正整数 N (N<=1000)和 M，其中 N 是图中结点个数，结点编号从 1 到 N； M（M<=10000）是边的数目。随后的 M 行中，每行给出一条边的信息，即该边连接的两个结点编号，中间用空格分隔。最后一行给出需要计算紧密度中心性的这组结点的个数 K（K<=100），以及 K 个结点的编号，用空格分隔。

（2）输出说明：按照“Cc(i)=x.xx”的格式输出 K 个给定结点的紧密度中心性，每个输出占一行，结果精确到小数点后 2 位。

（3）测试用例：

输入　5 8

1 2

1 3

1 4

2 3
3 4
4 5
2 5
3 5
2 4 3

输出 Cc(4) = 0.80
Cc(3) = 1.00

35. 公交线路提示

[问题描述]

上网下载所在城市真实公交线路图，建立所在城市主要公交线路图的存储结构。

[基本要求]

（1）输入任意两站点，给出转车次数最少的乘车路线。

（2）输入任意两站点，给出经过站点最少的乘车路线。

36. 连连看

[问题描述]

建立一个 10×20 的矩形方格图，其中有 10 种不同的图案，每种图案个数为偶数，填满矩形方格图。

[基本要求]

（1）随机产生原始数据；

（2）输入两个位置，如果两者图案相同，并且可以用小于等于 3 条直线相连，即可消除该两个图案。

6.10 延伸阅读

图论的第一篇著名论文“哥尼斯堡七桥问题”由 L. Euler 于 1736 年发表[1]。这篇论文中提出了有关图论奇数点、偶数点和欧拉图的定义，以及判断有无解的定理，开创了数学的一个新的分支－图论与几何拓扑，同时也给出了欧拉回路的求解算法。

Robert E. Tarjan 在其 1972 年发表的一篇文献中[2]详细介绍了深度优先搜索的回溯功能、其递归和非递归实现的便利性，以及在有向图的强连通分量和无向图的双连通分量求解中的应用。

1973 年图灵奖得主 John E. Hopcroft 和 Tarjan 共同发表了一篇关于求解图的连通分量和利用迭代求解简单路径的文章，在该文中提倡使用邻接表方式存储图，以提高求解算法的效率[3]。

匈牙利小说家、剧作家、诗人 Karinthy Frigyes 于 1929 年在其所著的短篇小说 Chains（*Láncszemek*）中首次提到了六度空间理论的概念。到了 20 世纪 60 年代，美国心理学家 Stanley Milgram 设计了一个连锁信件实验[4]，验证了在社会网络中任意两个人之间建立联系，最多间隔人数为 6 个人。现在人们普遍认为 Stanley Milgram 是六度空间理论的提出者。

求单源最短路径的 Dijksta 算法由 Edsger W. Dijkstra 在 1959 年提出[5]，求带负权值的单源最短路径的 Bellman-Ford 算法于 1954 年由 Alfonso Shimbel 提出[6]，而求所有顶点对之间最短路径的 Floyd-Warshall 算法于 1959 年由 Bernard Roy 提出[7]。

最小生成树的 Prim 算法于 1930 年由捷克数学家 Vojtěch Jarník 提出[8]，Kruskal 算法于 1956 年由 Joseph B. Kruskal 提出[9]。

拓扑排序算法由 Arthur B. Kahn 在 1962 年提出[10]。第一个关于关键路径分析的算法出现在 1959 年[11]。

参考文献：

[1] EULER L. Solutio problematis ad geometriam situs pertinentis[J]. Commentarii Academiae Scientiarum Petropolitanae, 1736, 8:128-140.

[2] TARJAN R E. Depth-first search and linear graph algorithms[J]. SIAM Journal on Computing, 1972, 1(2): 146-160.

[3] HOPCROFT J E, TARJAN R E, Efficient algorithms for graph manipulation[J]. Communications of the ACM, 1973, 16(6): 372-378.

[4] MILGRAM S. The small world problem[J]. Psychology Today, 1967, 2(1): 60-67.

[5] DIJKSTRA E W, A note on two problems in connexion with graphs[J]. Numerische Mathematik, 1959, 1(1): 269-271.

[6] SHIMBEL A. Structure in communication nets[C]. Proceedings of the Symposium on Information Networks. New York: Polytechnic Press of the Polytechnic Institute of Brooklyn, 1954: 199-203.

[7] ROY B. Transitivité et connexité[J]. Comptes Rendus de l'Academie, 1959, 249: 216-218

[8] JARNIK V. O jistém problému minimálním [About a certain minimal problem][J]. Práce moravské přírodovědecké společnosti (in Czech), 1930, 6 (4): 57-63.

[9] KRUSKAL J B. On the shortest spanning subtree of a graph and the traveling salesman problem[J]. Proceedings of the American Mathematical Society, 1956,7 (1): 48-50.

[10] KAHN A B. Topological sorting of large networks[J]. Communications of the ACM, 1962, 5 (11): 558-562.

[11] KELLEY J E, WALKER M R. Critical-path planning and scheduling[C]. Proceedings of the Eastern Joint IRE-AIEE-ACM Computer Conference, 1959: 160-173.